

INFROME FORENSE DE LAS IMAGENES DE LA CLONADORA – 5024_04-00

05/12/2025

Valdría hacerlo simplemente con E01, él mismo entiende que el resto es la misma unidad. Primero vamos a comprobar el estado de formato de la imagen forense con el comando `img_stat`.

```
marina@KaliMoxy: ~/Escritorio/Ciberseguridad/Analisis/Tarea2/PNoVolatiles/Clon/Imagen/2025-11-27_18-48-02
$ img_stat image.E01
IMAGE FILE INFORMATION
Image Type: ewf
Size of data in bytes: 7794589696
Sector size: 512
MD5 hash of data: e3d8e1d184d3a4a3404efb869773451

marina@KaliMoxy: ~/Escritorio/Ciberseguridad/Analisis/Tarea2/PNoVolatiles/Clon/Imagen/2025-11-27_18-48-02
$ img_stat image.E02
IMAGE FILE INFORMATION
Image Type: ewf
Size of data in bytes: 7794589696
Sector size: 512
MD5 hash of data: e3d8e1d184d3a4a3404efb869773451

marina@KaliMoxy: ~/Escritorio/Ciberseguridad/Analisis/Tarea2/PNoVolatiles/Clon/Imagen/2025-11-27_18-48-02
$ img_stat image.E03
IMAGE FILE INFORMATION
Image Type: ewf
Size of data in bytes: 7794589696
Sector size: 512
MD5 hash of data: e3d8e1d184d3a4a3404efb869773451

marina@KaliMoxy: ~/Escritorio/Ciberseguridad/Analisis/Tarea2/PNoVolatiles/Clon/Imagen/2025-11-27_18-48-02
$
```

Esto nos muestra el tipo de imagen, el tamaño en bytes, el sector del tamaño y el hash en MD5 del dato. El hash nos tendría que salir igual que en el log que nos salió con la clonadora. A continuación, utilizamos el comando `img_cat` para comprobar el contenido de la imagen forense sin procesar.

```
marina@KaliMoxy: ~/Escritorio/Ciberseguridad/Analisis/Tarea2/PNoVolatiles/Clon/Imagen/2025-11-27_18-48-02
$ img_cat image.E01 | head -10
**xmkfs.fs fa **xmkfs** p#3NO NAME FAT32 *u/**t
V*****2*****This is not a bootable disk.
Please insert a bootable floppy and
press any key to try again ...
U#RRARra#*Q#**xmkfs.fs fa **xmkfs** p#3NO NAME FAT32 *u/**t
V*****2*****This is not
a bootable disk. Please insert a bootable floppy and
press any key to try again ...
U#RRARra#*Q#**xmkfs.fs fa **xmkfs**

[!]"$%&'()*+,-./0123456789:;=<?@*~#BCDEFHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~
.....

[!]"$%&'()*+,-./0123456789:;=<?@*~#BCDEFHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~
.....
*****z|}|}*****C32289*0.jpg***27130*5-502758photo-1609308PH
OTO-1JPG V**y[y[*]([Ebp*****educ*ativo-we-coma-recursotil-lea:imagen*Por-
que-es-u#ROR-Q-1WEB **y[y[*]([E*Almage:s.jpg***IMAGES JPG **y[y[*]([A*0B485.jpg.webphal
ia-lite-BONALITE-1WEB **y[y[*]([D*IDpg*****eser f#orens#Jstud#ar para que h#
ay que eQUEHAV-1JPG
**y[y[*]([Z*#ENOLOG*IA.jpg*#ERTO-#EN-CRIMISE-MA*STER-EXPCOLOG*IA-FORENMAST
E#-EN-PSIMASTER-1JPG **y[y[*]([y#0Dpg*****30.or*iginal.jorens#e-1200*opsiqu#
latria-FPSIQUI-1JPG c**y[y[*]([c*#***3FIFHH**
XICC_PROFILE
HLInomtrRGB XYZ
RGB***HP cprtP3desc*lwtp*bkptXYZ2@XYZ,bXYZBdmndTpdmd*vueld*view*slumi*neas
$tech$
rTRC
gTRC
x
bTRC
textCopyright (c) 1998 Hewlett-Packard CompanydescsRGB IEC61966-2.1sRGB IEC61966-2.1XYZ *Q
*XYZ XYZ o#**XYZ b***XYZ $****descIEC http://www.iec.chIEC http://www.iec.chdesc.IEC 61966-
2.1 Default RGB colour space - sRGB.IEC 61966-2.1 Default RGB colour space - sRGBdesc.Referenc
e Viewing Condition in IEC61966-2.1Reference Viewing Condition in IEC61966-2.1view***\xyz
Z L VPMWag*sig CRT curv
x#20#ELRY gnu*****
b(BAKT)gqz*****
+IXgw*****7HYJ{*****=Out***** % !-8C0Zfr-*****-jHUCq-*****
* * * * *
.

marina@KaliMoxy: ~/Escritorio/Ciberseguridad/Analisis/Tarea2/PNoVolatiles/Clon/Imagen/2025-11-27_18-48-02
$
```

Esta en bruto y no podemos verlo adecuadamente. Hemos utilizado el “head -10” para solo visualizar las diez primeras líneas. No más porque es un contenido muy largo. Si queremos encontrar algo en específico, es posible que queramos verlo entero. Sale lo mismo en las otras dos.

Continuamos con el comando `mmls` que muestra el diseño del disco del que se sacó la imagen. Saldrán incluso los espacios no asignados.

```
marina@KaliMoxy: ~/Escritorio/Ciberseguridad/Analisis/Tarea2/PHoVolatiles/Clon/Imagen/2025-11-27_18-48-02
Session Acciones Editar Vista Ayuda

marina@KaliMoxy:~/Escritorio/Ciberseguridad/Analisis/Tarea2/PHoVolatiles/Clon/Imagen/2025-11-27_18-48-02$ mmls image.E01
GUID Partition Table (EFI)
Offset Sector: 0
Units are in 512-byte sectors

Slot      Start      End      Length  Description
000:      0000000000  0015223807  0015223808  Unallocated
001: Meta   0015223775  0015223806  0000000032  Partition Table
002: Meta   0015223807  0015223807  0000000001  GPT Header

marina@KaliMoxy:~/Escritorio/Ciberseguridad/Analisis/Tarea2/PHoVolatiles/Clon/Imagen/2025-11-27_18-48-02$ mmls image.E02
GUID Partition Table (EFI)
Offset Sector: 0
Units are in 512-byte sectors

Slot      Start      End      Length  Description
000:      0000000000  0015223807  0015223808  Unallocated
001: Meta   0015223775  0015223806  0000000032  Partition Table
002: Meta   0015223807  0015223807  0000000001  GPT Header

marina@KaliMoxy:~/Escritorio/Ciberseguridad/Analisis/Tarea2/PHoVolatiles/Clon/Imagen/2025-11-27_18-48-02$ mmls image.E03
GUID Partition Table (EFI)
Offset Sector: 0
Units are in 512-byte sectors

Slot      Start      End      Length  Description
000:      0000000000  0015223807  0015223808  Unallocated
001: Meta   0015223775  0015223806  0000000032  Partition Table
002: Meta   0015223807  0015223807  0000000001  GPT Header

marina@KaliMoxy:~/Escritorio/Ciberseguridad/Analisis/Tarea2/PHoVolatiles/Clon/Imagen/2025-11-27_18-48-02$
```

Seguimos con el `mmstat` que muestra detalles sobre un dispositivo, que será del que hicimos la imagen. Normalmente solo aparece el tipo del dispositivo.

```
marina@KaliMoxy: ~/Escritorio/Ciberseguridad/Analisis/Tarea2/PHoVolatiles/Clon/Imagen/2025-11-27_18-48-02
Session Acciones Editar Vista Ayuda

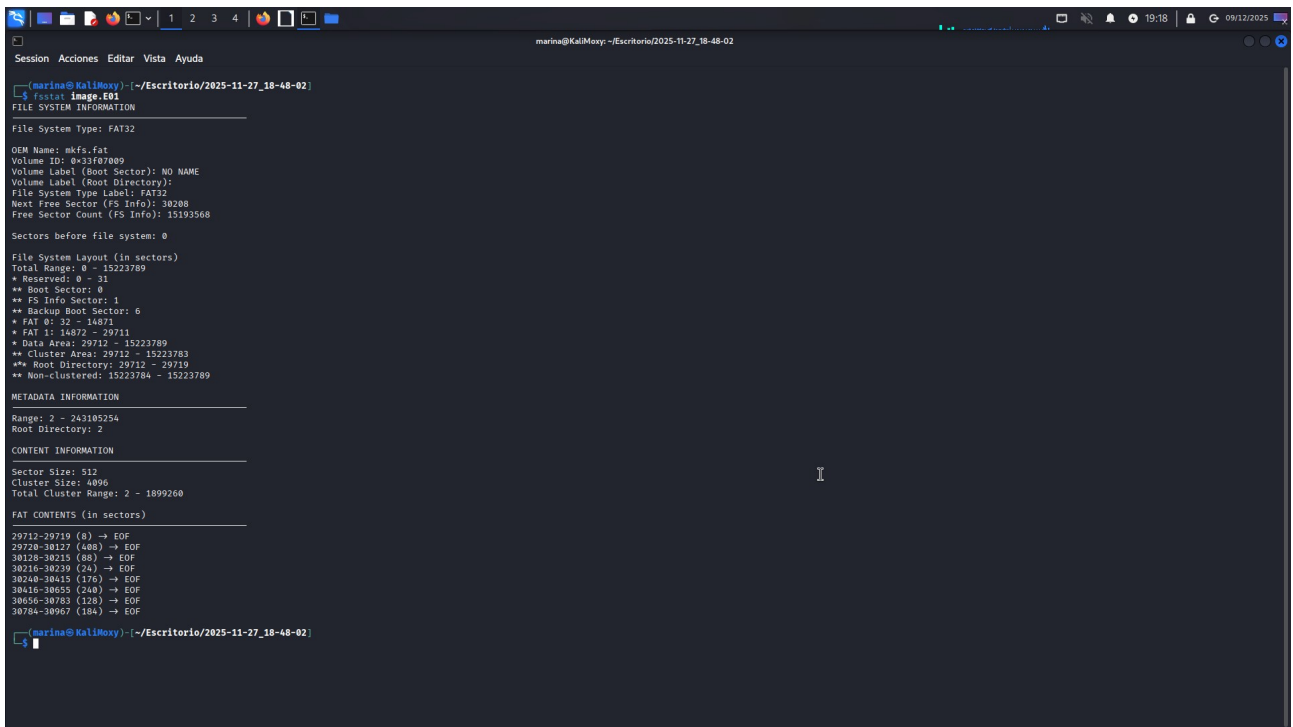
marina@KaliMoxy:~/Escritorio/Ciberseguridad/Analisis/Tarea2/PHoVolatiles/Clon/Imagen/2025-11-27_18-48-02$ mmstat image.E01
gpt

marina@KaliMoxy:~/Escritorio/Ciberseguridad/Analisis/Tarea2/PHoVolatiles/Clon/Imagen/2025-11-27_18-48-02$ mmstat image.E02
gpt

marina@KaliMoxy:~/Escritorio/Ciberseguridad/Analisis/Tarea2/PHoVolatiles/Clon/Imagen/2025-11-27_18-48-02$ mmstat image.E03
gpt

marina@KaliMoxy:~/Escritorio/Ciberseguridad/Analisis/Tarea2/PHoVolatiles/Clon/Imagen/2025-11-27_18-48-02$
```

En este caso nos muestra que el tipo de dispositivo del que se hizo la imagen, era `gpt`. Con el `mmcat`, extraemos el conenido de un volumen específico a `STODOOUT`. `Mmcat` es una utilidad de la capa de gestión de medios/volumenes de TSK. Su función principal es extraer el contenido bruto de una partición de una imagen de disco. Es como volcar el contenido de la particiõn. Esto permite redirigir la salida a un archivo, de modo que obtengas una imagen separada de esa particiõn. Lo cual es ideal para analizar luego con herramientas de sistemas de archivos.



```
marina@KaliMoxy: ~/Escritorio/2025-11-27_18-48-02
$ faststat image.E01
FILE SYSTEM INFORMATION
-----
File System Type: FAT32
OEM Name: mkfs.fat
Volume ID: 0x33f07009
Volume Label (Boot Sector): NO NAME
Volume Label (Root Directory):
File System Type Label: FAT32
Next Free Sector (FS Info): 30208
Free Sector Count (FS Info): 15193568
Sectors before file system: 0
File System Layout (in sectors)
Total Range: 0 - 15223789
* Reserved: 0 - 31
** Boot Sector: 0
** FS Info Sector: 1
** Backup Root Sector: 6
* FAT 0: 32 - 14871
* FAT 1: 14872 - 29711
* Data Area: 29712 - 15223789
** Cluster Area: 29712 - 15223783
** Root Directory: 29712 - 29719
** Non-clustered: 15223784 - 15223789
METADATA INFORMATION
-----
Range: 2 - 243105254
Root Directory: 2
CONTENT INFORMATION
-----
Sector Size: 512
Cluster Size: 4096
Total Cluster Range: 2 - 1899268
FAT CONTENTS (in sectors)
-----
29712-29719 (8) -> EOF
29720-30227 (408) -> EOF
30228-30235 (8) -> EOF
30236-30239 (4) -> EOF
30240-30415 (176) -> EOF
30416-30655 (240) -> EOF
30656-30783 (128) -> EOF
30784-30967 (184) -> EOF
```

Con esta herramienta hemos conseguido visualizar: el tipo de sistemas de archivos que utilizaba el dispositivo de la imagen, el diseño, los tamaños y las etiquetas. En esta imagen podemos ver la información del sistema de archivos, los metadatos, el contenido y los contenidos en sectores.

En concreto en esta imagen, podemos ver que utiliza FAT32 que sistema de archivos. Indica el nombre OEM, siendo este mkfs.fat, lo que significa que fue creado con la utilidad de Linux para formatear FAT. También se visualiza el identificador del volumen y la etiqueta que puede tener el disco, en este dispositivo no tiene nombre. Además, se visualiza la información de dónde está el próximo sector libre y cuántos sectores libres hay según la información del sistema de archivos.

En esta parte también se encuentra la ubicación de la estructura del sistema de archivos. En este caso, no hay sectores antes del sistema de archivos, es decir, la partición comienza al inicio de la imagen. Y muestra el layout del sistema de archivos y sus sectores. Muestra los reservados (Reserved) que son los que se reservan para la información crítica del sistema de archivos y cada uno de sus sectores (Boot, FS info, Backup Boot). Después tenemos los sectores FAT0 y FAT1 que son las tablas de asignación de archivos, necesarias para FAT32. Y por último, tenemos el Data Area que es donde se almacenan los datos. Tenemos los datos raíz (Root Directory) y los sectores fuera de clusters normales (Non-clustered).

Después mostraremos la información de los datos, que muestra la gama de clusters para metadatos y ubicación del root directory. A continuación, se muestra la información de contenido. Cada sector tiene 512 bytes, cada cluster son 8 sectores (4096 bytes). Además, hay 1,899,259 clusters en total en el área de datos.

Por último, se muestran los contenidos de FAT. Cada línea indica un bloque de clusters y a dónde apunta según la FAT. El “EOF” significa que esos clusters están al final de un archivo. FAT32 utiliza esa tabla para rastrear qué clusters pertenecen a qué archivos.

Se utiliza el jcat y jls como herramientas del journaling del sistema de archivos. El jcat muestra el contenido de un bloque de un *journaling* específico. Mientras que el jls enumera las entradas en el *journaling* del sistema de archivos.

```
marina@KaliMox: ~/Escritorio/2025-11-27_18-48-02
Session Acciones Editar Vista Ayuda

marina@KaliMox:~/Escritorio/2025-11-27_18-48-02$ ./ls -f fat32 image.E01
Inode value is too small for image (2)

marina@KaliMox:~/Escritorio/2025-11-27_18-48-02$ ./cat -f fat32 image.E01 0
Inode value is too small for image (2)

marina@KaliMox:~/Escritorio/2025-11-27_18-48-02$
```

Nos da un error esperado. Estos comandos esperan un sistemas de archivos con journal, cosa con la que no cuenta FAT32. Al indicarle “-f fat32” TSK intenta interpretar la imagen como si FAT32 tuviera journal. Es básicamente un error de diseño, en lugar de decir que FAT32 no tiene journal, TSK intenta buscar estructuras y falla. El error real es que estamos intentado utilizar una herramienta de journal en un sistema de archivos sin journal.

Ahora vamos a utilizar las herramientas de nombre de archivo. En esta utilizaremos primero fls para enumerar los nombres de archivos asignados y eliminados en un directorio.

```
marina@KaliMox: ~/Escritorio/2025-11-27_18-48-02
Session Acciones Editar Vista Ayuda

marina@KaliMox:~/Escritorio/2025-11-27_18-48-02$ fls -r -i ewf image.E01
r/r 6: photo-1689308271305-50e75832289b.jpg
r/r 12: ¿Por-que-es-util-la-imagen-como-recurso-educativo.webp
r/r 14: images.jpg
r/r 17: hail-hi-s-00485.jpg.webp
r/r 22: que hay que estudiar para ser forense.jpg
r/r 28: MASTER-EN-PSICOLOGIA-FORENSE-MASTER-EXPERTO-EN-CRIMINOLOGIA.jpg
r/r 31: psiquiatria-forense-1200x630-original.jpg
v/v 243105251: $MBR
v/v 243105252: $PAT1
v/v 243105253: $FAT2
V/V 243105254: $OrphanFiles
+ -/r * 16697231: H""L""H"".$*
+ -/d * 158666487: M"""""""".$*
+ -/r * 163619923: """""""""".$*
+ -/r * 163757811: EFISECTOR
+ -/d * 163757812: EFI
+ -/d * 163758037: BOOT
+ -/r * 163758053: BOOTX64.EFI
+ -/r * 163803891: EFISECTOR
+ -/d * 163803892: EFI
+ -/d * 163804117: BOOT
+ -/r * 163804131: BOOTX64.EFI
+ -/r * 163922147: H""L""H"".$*

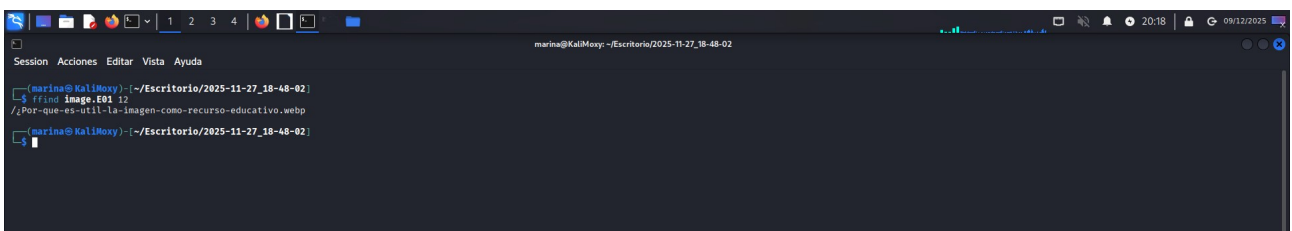
marina@KaliMox:~/Escritorio/2025-11-27_18-48-02$
```

Con este comando listamos los archivos y directorios dentro de un sistema de archivos. Con “-r” decimos que sea recursivo y muestre todo dentro de subdirectorios. Con el “-i” indicamos el tipo de imagen que es, en este caso ewf. Y por último pondemos el nombre de la imagen a analizar.

Nos muestra con “r/r” archivos regulados no borrados. El primer número es el inode de los archivos. Por ejemplo, en este caso “¿Por-que-es-util-la-imagen-como-recurso-educativo.webp” tiene el inode 12. O el archivo “images.jpg” tiene el inode 14.

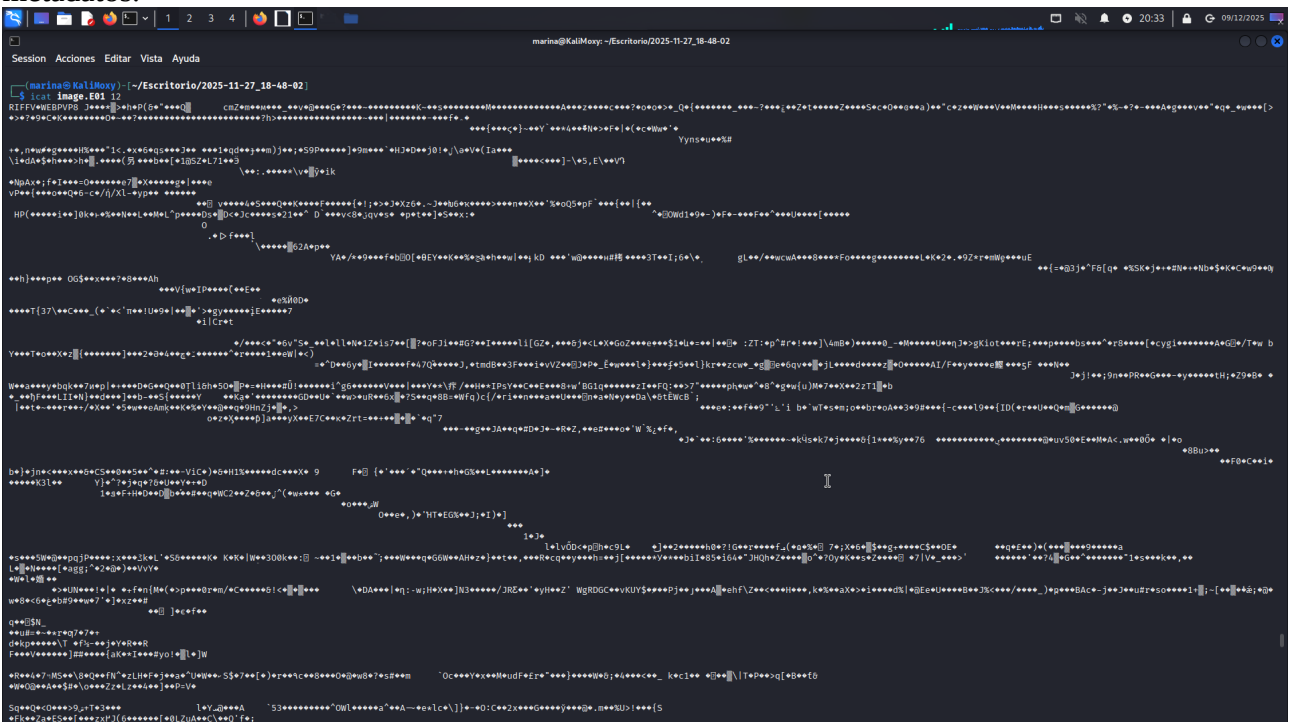
Con “v/v” o “V/V” se muestran los archivos especiales y del sistema. En minúscula “v/v” son archivos virtuales del sistema de archivos, por ejemplo las áreas del FAT. Y con “V/V” se muestran los grandes archivos virtuales, como “OrphanFiles” que son archivos huérfanos no referenciados en directorios. Los números largos como el 243105254 del \$OrphanFiles es el inode de los archivos. Los que empiezan con “+ -/” son archivos borrados y fragmentados. El “+” indica que el archivo está fragmentado, el “-/” referencia a un archivo regular fragmentado, mientras que el “-/d” representa archivos borrados. El “*” es que el inode es desconocido o no asignado, porque fls no puede asignárselo a un archivo existente. Los número grandes son la ubicación de inicio del fragmento, como el 163757811 del primer “EFISECTOR”. Los símbolos y letras del principio y el final, como H[^]L[^]H[^]\$[^], son el contenido parcial leído como texto el cual no es legible normalmente. Para analizar correctamente estas partes ilegibles, se deberían extraer los datos binarios con herramientas como icat, no leerlos en pantalla.

Con el `ffind` buscamos los nombres de archivos asignados y no asignados que apuntan a una estructura de metadatos determinada. Por ejemplo en este caso, vamos a buscar el nombre del archivo del inode 12.



En esta imagen nos muestra que el nombre del archivo del inode 12 es “¿Por-que-es-util-la-imagen-como-recurso-educativo.webp”.

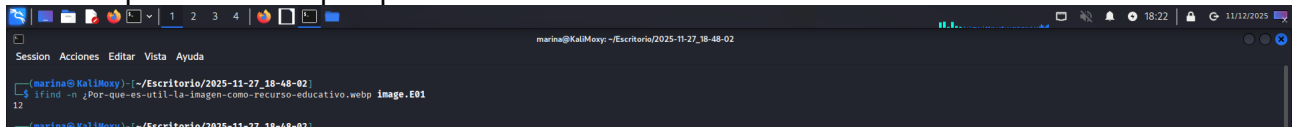
Continuamos con las herramientas de metadatos en i-nodo. En este caso, vamos a empezar con el icat que extrae las unidades de datos de un archivos, que se especifica por su dirección de metadatos.



En este comando nos muestra el contenido del archivo inode 12 de la imagen E01 y lo muestra en stdout. El inde 12 corresponde a “¿Por-que-es-util-la-imagen-como-recurso-educativo.webp”. Por lo que estamos viendo el contenido binario de un archivo .webp directamente en la terminal. Por ello no se muestran caracteres legibles.

11/12/2025

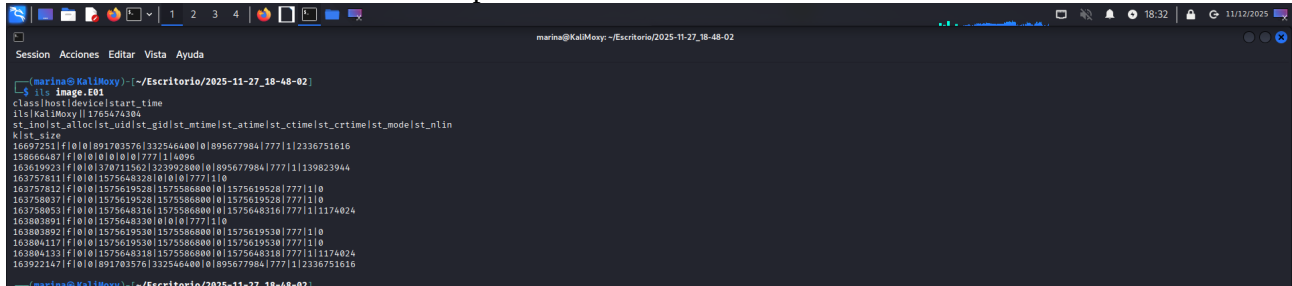
Con el comando de ifind podemos buscar la estructura de metadatos que tiene un nombre de archivo determinado. Por ejemplo, en este caso podemos realizar un ifind al mismo archivo que antes y nos muestra que el inodo al que apunta es el 12.



```
marina@KaliMoxy: ~/Escritorio/2025-11-27_18-48-02
$ ifind -n ¿Por-que-es-util-la-imagen-como-recurso-educativo.webp image.E01
12
```

TSK ha encontrado el inodo asociado a ese nombre de archivo, y su nombre de inodo es 12. “ifind -n” busca un archivo por el nombre y ha devuelto el inodo del archivo.

A continuación se ha probado el comando ils que enumera las estructuras de metadatos y su contenido en un formato delimitado por tuberías.



```
marina@KaliMoxy: ~/Escritorio/2025-11-27_18-48-02
$ ils image.E01
class|host|device|start_time
ils|KaliMoxy||1765474384
st_ino|st_alloc|st_uid|st_gid|st_mtime|st_atime|st_ctime|st_mode|st_nlink
kfst_size
16697251|f|0|0|891703576|332546400|0|895677984|777|1|2336751616
15866487|f|0|0|0|0|777|1|14096
163619923|f|0|0|370711562|323992800|0|895677984|777|1|139823944
163757811|f|0|0|1575648328|0|0|777|1|0
163757812|f|0|0|1575619528|1575586800|0|1575619528|777|1|0
163758037|f|0|0|1575619528|1575586800|0|1575619528|777|1|0
163758053|f|0|0|1575648316|1575586800|0|1575648316|777|1|1174024
163803891|f|0|0|1575648330|0|0|777|1|0
163803892|f|0|0|1575619530|1575586800|0|1575619530|777|1|0
163804117|f|0|0|1575619530|1575586800|0|1575619530|777|1|0
163804133|f|0|0|1575648331|1575586800|0|1575648331|777|1|1174024
163922147|f|0|0|891703576|332546400|0|895677984|777|1|2336751616
```

En esta captura se puede ver el resultado del comando ils. Las dos primeras líneas hacen referencia al encabezado de la tabla principal, donde se especifica el tipo de comando o módulo usado, el nombre de la máquina donde se hizo el volcado, el dispositivo y la marca de tiempo del inicio del análisis.

Después sigue el encabezado de metadatos de inodo: número de inodo, si está aislado o no, id del usuario propietario, id del grupo propietario, hora de la última modificación, hora del último acceso, hora de la última modificación, hora de creación, permisos del archivo, número de enlaces al archivo y tamaño en bytes del archivo.

Un ejemplo de lo que sale por pantalla es: 16697251|f|0|0|891703576|332546400|0|895677984|777|1|2336751616 . Analicemos esta línea. Primero se nos muestra el número del inodo (16697251). A continuación se nos muestra el inodo asignado (f), y el propietario y el grupo es root (0|0). Después nos muestra la hora de la última modificación (891703576), que al pasarlo a fecha real queda como 1998-04-23 08:26:16 UTC. También se nos muestra el último acceso (332546400) que habría que pasarlo a tiempo real 1980-07-02 03:20:00 UTC. Nos muestra con el 0 que no hay registrado un cambio de metadatos. En el otro número (895677984) nos muestra la hora de creación que es aproximadamente 1998-06-04 15:36:24 UTC. Los tres siguientes nos muestra que tiene los permisos completos y el 1 indica que tiene solo un enlace al archivo. El último número es el tamaño en bytes (2336751616), que sería más o menos 2,18GB.

12/12/2025

El comando istat muestra las estadísticas y los detalles sobre una estructura de metadatos determinada en n formato que es fácil de leer. Habría que poner el parámetro “-f” para especificar el sistema de fichero de datos, el nombre de la imagen y el inodo del que queremos investigar.


```
Session Acciones Editar Vista Ayuda
$ ls -lstat -f fat image.E01 6
Directory Entry: 6
Allocated:
File Attributes: File, Archive
Size: 207026
Name: PHOTO~1.JPG
Directory Entry Times:
Written: 2025-11-25 19:28:24 (CET)
Accessed: 2025-11-25 00:00:00 (CET)
Created: 2025-11-25 19:28:24 (CET)
Sectors:
29720 29721 29722 29723 29724 29725 29726 29727
29728 29729 29730 29731 29732 29733 29734 29735
29736 29737 29738 29739 29740 29741 29742 29743
29744 29745 29746 29747 29748 29749 29750 29751
29752 29753 29754 29755 29756 29757 29758 29759
29760 29761 29762 29763 29764 29765 29766 29767
29768 29769 29770 29771 29772 29773 29774 29775
29776 29777 29778 29779 29780 29781 29782 29783
29784 29785 29786 29787 29788 29789 29790 29791
29792 29793 29794 29795 29796 29797 29798 29799
29800 29801 29802 29803 29804 29805 29806 29807
29808 29809 29810 29811 29812 29813 29814 29815
29816 29817 29818 29819 29820 29821 29822 29823
29824 29825 29826 29827 29828 29829 29830 29831
29832 29833 29834 29835 29836 29837 29838 29839
29840 29841 29842 29843 29844 29845 29846 29847
29848 29849 29850 29851 29852 29853 29854 29855
29856 29857 29858 29859 29860 29861 29862 29863
29864 29865 29866 29867 29868 29869 29870 29871
29872 29873 29874 29875 29876 29877 29878 29879
29880 29881 29882 29883 29884 29885 29886 29887
29888 29889 29890 29891 29892 29893 29894 29895
29896 29897 29898 29899 29900 29901 29902 29903
29904 29905 29906 29907 29908 29909 29910 29911
29912 29913 29914 29915 29916 29917 29918 29919
29920 29921 29922 29923 29924 29925 29926 29927
29928 29929 29930 29931 29932 29933 29934 29935
29936 29937 29938 29939 29940 29941 29942 29943
29944 29945 29946 29947 29948 29949 29950 29951
29952 29953 29954 29955 29956 29957 29958 29959
29960 29961 29962 29963 29964 29965 29966 29967
29968 29969 29970 29971 29972 29973 29974 29975
29976 29977 29978 29979 29980 29981 29982 29983
29984 29985 29986 29987 29988 29989 29990 29991
29992 29993 29994 29995 29996 29997 29998 29999
30000 30001 30002 30003 30004 30005 30006 30007
30008 30009 30010 30011 30012 30013 30014 30015
30016 30017 30018 30019 30020 30021 30022 30023
30024 30025 30026 30027 30028 30029 30030 30031
30032 30033 30034 30035 30036 30037 30038 30039
30040 30041 30042 30043 30044 30045 30046 30047
30048 30049 30050 30051 30052 30053 30054 30055
30056 30057 30058 30059 30060 30061 30062 30063
30064 30065 30066 30067 30068 30069 30070 30071
30072 30073 30074 30075 30076 30077 30078 30079
30080 30081 30082 30083 30084 30085 30086 30087
30088 30089 30090 30091 30092 30093 30094 30095
30096 30097 30098 30099 30100 30101 30102 30103
```

La conclusión inicial es que es un archivo jpg asignado, que no ha sido eliminado, on fragmentado, con metadatos FAT coherentes. Además, ocupa exactamente los sectores que se han listado. Con esto, podemos realizar unos cálculos teniendo en cuenta que se indica que ocupa 207026 bytes. Un sector en FAT tiene 512 bytes. Entonces se necesita 405 sectores. Y el último sector que se muestra, que no cabe en la imagen, es 30120 30121 30122 30123 30124 0 0 0, los ceros indican que no es espacio no usado. Coincide exacta,emte los sectores asignados son los necesarios para alojar el archivo.

La lista larga son los sectores, se puede observar que es una secuencia creciente y continua. Esto indica que el archivo no es fragmentado.

Las líneas Created o Written indican que el archivo se creó el 25/11/2025 a la hora indicada 19:28:24. Esto indica que el archivo se creó y no se modificó después, pues la fecha de creación y la última escritura es lo mismo. Accessed muestra un acceso sin hora, es un comportamiento típico de FAT. No es indicativo de manipulación por sí mismo. Estos datos indican que no se muestra ninguna evidencia entre Created/Modified que sugieran alteración intencional.

PHOTO~1.JPG es un nombre 8.3 (short name). Esto indica que el nombre del fichero original es más largo del que se muestra. La extensión .jpg sugiere imagen jpeg.

También se pueden destacar algunos riesgos. El slack de 334 bytes del último sector puede contener restos de datos que podrían ser relevantes. El atributo Archive no implica nada por sí solo. Puesto que está en Allocated, no necesitamos técnicas para recuperarlo, basta con extraerlo.

Podemos continuar con las herramientas de contenido (datos). Para eso primero vamos a utilizar blkcat que extrae el contenido exacto de una unidad de datos (bloque) específica.

15/12/2025